

Коваль Олександр Дмитрович
2ПІ-25Б
Тема: Roman Conquest

ВСТУП

Сучасний етап розвитку людської цивілізації нерозривно пов'язаний із глобальною цифровізацією та стрімким впровадженням інноваційних технологій у всі сфери нашої життєдіяльності. Досягнення сучасних інформаційних технологій (ІТ) здійснили справжню революцію з точки зору збереження інформації, її передачі на великі відстані, математичного та логічного обрахунку у реальному часі, а також високоякісної візуалізації, в тому числі тривимірної. Людство здійснило колосальний стрибок від розрізнених паперових архівів до багаторівневих реляційних та об'єктно-орієнтованих баз даних, хмарних сховищ, які здатні надійно акумулювати терабайти історичних, статистичних та структурних відомостей, гарантуючи їхню стійкість до втрат. Розвиток глобальних оптоволоконних та бездротових мереж дозволяє обмінюватися цими масивами даних практично миттєво, стираючи будь-які географічні межі. Проте одним із найважливіших досягнень є здатність сучасних обчислювальних систем та парадигм програмування (зокрема об'єктно-орієнтованого підходу) обробляти складні алгоритми у реальному часі. Використання багатопотоковості дозволяє системі одночасно керувати станами тисяч незалежних програмних сутностей. У поєднанні з передовими графічними рушіями, це дає змогу відтворювати гіперреалістичні симуляційні світи з деталізованою фізикою, де кожен елемент функціонує за наперед заданими правилами.

Незважаючи на цей безпрецедентний технологічний прогрес, сучасне суспільство стикається з глибокою соціокультурною та освітньою кризою, що набуває ознак глобальної проблеми. Складається враження, що люди починають все менше звертати увагу на історію Давнього Риму, його фундаментальну культуру та неоціненний внесок у формування сучасної європейської та світової цивілізації. Світ охоплюють своєрідні інформаційні жахи сьогодення: молоде покоління стрімко втрачає історичну пам'ять, страждає від так званого «кліпового мислення» та повністю розсіює власну увагу в нескінченних потоках низькоякісного, швидкого розважального контенту. Суспільство поступово

забуває, що Давній Рим славився не тільки видатними філософами, талановитими архітекторами, правознавцями і науковцями, а й, на свій час, найбільш розвинутою та ефективною мілітаризованою сферою. Це була система з надзвичайно чіткою, бездоганно налагодженою військовою та суспільною ієрархією, залізною дисципліною та передовим стратегічним плануванням, яке дозволило побудувати одну з найвеличніших імперій в історії. Державні інституції та традиційні освітні заклади продовжують нести величезні витрати коштів на друк консервативних підручників та підтримку застарілих методик викладання. Проте ці колосальні фінансові вливання не дають бажаного результату, викликаючи у сучасній аудиторії лише нудьгу та відторгнення. Це реальна проблема, яку неможливо подолати класичними інструментами, але яку можна успішно вирішити шляхом інтерактивного комп'ютерного моделювання і прогнозування, адаптувавши подачу інформації до вимог цифрової епохи.

Можливість застосування сучасних ІТ у проблемі, що розглядається, відкриває абсолютно нові, безмежні горизонти для інноваційних освітніх підходів. Замість пасивного споживання сухого текстового матеріалу, ми маємо унікальну можливість використати всю потужність об'єктно-орієнтованого програмування для створення інтерактивної форми ознайомлення. Розробка динамічної стратегічної гри або навчального симулятора дозволяє перевести складний історичний контекст у зрозумілу та захопливу площину. Програмне середовище надає інструментарій для детального моделювання архітектури, економіки та життєдіяльності римського військового табору. З інфраструктурної точки зору, така програмна система органічно спирається на глобальні макрооб'єкти: «Джерело руди» забезпечуватиме економічне підґрунтя та критично важливу ресурсну базу для розвитку, центральна «База» відповідатиме за логістику, управління та генерацію нових одиниць, а «Башта» формуватиме неприступний оборонний периметр та надаватиме тактичну перевагу. З іншого боку, деталізація ігрових процесів реалізовуватиметься через мікрооб'єкти, які ідеально репрезентують римську військову ієрархію: від базового «Воїна» (легіонера), через тактично підготовленого «Центуріона», і аж до елітного «Преторіанця». Користувач отримує

можливість безпосередньо взаємодіяти з цією багаторівневою системою, керувати розподілом ресурсів, розробляти власні стратегії та аналізувати наслідки прийнятих рішень. Через такий захопливий ігровий процес молодь підсвідомо засвоює складні принципи римської організації, що робить вивчення історії максимально ефективним та живим.

З огляду на нагальну необхідність подолання глибокої кризи історичної необізнаності, а також враховуючи колосальний потенціал сучасних інструментів об'єктно-орієнтованого програмування для створення інтерактивних навчально-ігрових симуляцій, саме тому в якості теми курсової роботи було обрано розробку програмної системи моделювання об'єктів гри "Roman Conquest".

1 АНАЛІЗ СУЧАСНОГО СТАНУ ПИТАННЯ ТА ОБҐРУНТУВАННЯ ЗАВДАННЯ НА РОБОТУ

1.1 Предметна область

Давній Рим залишається однією з найвпливовіших цивілізацій в історії людства, чий досягнення у військовій справі, державному управлінні та культурі заклали фундамент сучасної європейської цивілізації. Римська армія вважається одним із найдосконаліших військових механізмів античності, що забезпечило створення і підтримку однієї з найбільших імперій у світовій історії. Проте сучасне покоління все менше цікавиться історією, особливо її військовими аспектами, віддаючи перевагу швидкому розважальному контенту замість глибокого вивчення історичних подій. Сучасні технології відкривають нові можливості для вивчення та популяризації історії через інтерактивні графічні програми, які дозволяють у захоплюючій формі демонструвати взаємодію макро- та мікрооб'єктів один з одним.

Військова організація Давнього Риму базувалася на чіткій ієрархічній структурі, де кожен воїн мав своє місце та функції. Кожна війна у античному світі не відбувалась без **воїнів (Warrior)**, які складали основу легіонів. Згідно з відомим висловом Отто фон Бісмарка, війни виграють не лише генерали, а й звичайні люди, які беруть у них участь. У римській армії воїни часто приходили на військову службу не з власного бажання, а через необхідність захищати інтереси Римської імперії на численних фронтах. Ці люди були змушені піти на війну і битися за Римську Імперію, формуючи хребет римської військової машини. Воїни брали участь у кампаніях на Заході та Сході, воювали з галами в глибинах Іспанії, переправлялися через Середземне море для боротьби з Карфагеном. Саме вони виконували накази командирів та брали участь у найважчих битвах.

Після багатьох років служби та участі в численних кампаніях, коли воїн провів достатньо військових операцій і здобув необхідний досвід та гроші, найздібніші з них могли отримати підвищення у ієрархії римського легіону і стати **центуріонами (Centurio)**. Це звання походить від латинського слова, що означає

десяток. Ця людина отримувала у свою владу вже десяток людей і повагу, краще спорядження та титул. Центуріони відмітилися в історії Риму як вправні воїни та мудрі командувачі, вони є надважливою ланкою у ході бою, на плечах яких лежав величезний обсяг завдань. Центуріони відігравали ключову роль у римській армії, адже на них лежала відповідальність за підготовку солдатів, підтримання дисципліни та прийняття тактичних рішень безпосередньо на полі бою. Вони отримували більшу повагу з боку товаришів та вищу оплату. Їхня майстерність володіння мечем та пілумом, стратегічне мислення та лідерські якості робили їх незамінними у військовій ієрархії.

Як тільки центуріон відзначився у бою, зробив героїчний вчинок, став найкращим майстром меча та пілума у своєму легіоні або врятував своїх людей від безвихідного оточення, йому пропонували стати **преторіанцем (Praetorian)**. Найвищою ланкою в ієрархії римських воїнів були преторіанці. Цих елітних воїнів відбирали серед найкращих солдатів з усієї імперії, і їхня задача була найскладнішою – захищати консулів та імператора. Преторіанці – це воїни при імператорі, це його особиста армія, розташована безпосередньо в Римі. Бути преторіанцем означало мати значний титул і перебувати у відносній безпеці навіть протягом довгих воєн, тому в програмі на тему Roman Conquest вони є найвищою ланкою ієрархії. Преторіанці отримували значно вищу платню порівняно зі звичайними легіонерами та мали особливі привілеї.

Проте успіх будь-якої імперії залежить не лише від сили її армії, а й від економічної основи. Кожна імперія була побудована насиллям, проте для існування імперії, утримання регулярної армії та побудови нових колоній необхідні гроші. Римська імперія була побудована на системі ресурсного забезпечення, де ключову роль відігравали рудники. **Джерело руди (Source)** є спорудою та макрооб'єктом, що приносить руду до команди гравця. Воно може бути захоплене будь-якою командою, захопити його може будь-яка ланка із ієрархії мікрооб'єктів, але в залежності від навичок конкретного воїна час захоплення може змінюватися. Контроль над джерелами руди означав контроль над виробництвом зброї,

інструментів та монет, необхідних для функціонування держави. Ці стратегічні об'єкти часто ставали предметом конфліктів між різними фракціями.

На шляху до бази імперії зустрічаються перешкоди. Тому в якості інтерпретації цього в програмі було обрано **башту (Tower)** як макрооб'єкт, що являє собою перешкоду для ворожої армії, яку необхідно зруйнувати щоб отримати шлях до бази супротивника. Оборонні башти виконували кілька важливих функцій. По-перше, вони забезпечували захищену позицію для оборонців. По-друге, башти слугували пунктами спостереження, звідки можна було виявити наближення ворога. По-третє, вони використовувалися як бази для постачання. Башти – це споруди, де мікрооб'єкти-союзники можуть отримати допомогу у вигляді поповнення очок здоров'я, також вони атакуватимуть найближчого ворожого мікрооб'єкта в певній області. Водночас ворожі башти становили загрозу, оскільки їхні захисники могли атакувати римських солдатів. Важливою особливістю башт є можливість полагодити їх, але зробити це може тільки центуріон завдяки набутому досвіду в ході кампаній та застосуванню його в реаліях битви.

Центральним елементом військової організації була **база (Base)** або головний табір. База є головною спорудою гравця, її треба захищати будь-якою ціною, оскільки база є початковою точкою всіх мікрооб'єктів будь-якої команди. Римські військові табори були справжніми шедеврами інженерної думки, де кожен елемент мав своє призначення. База служила відправною точкою для всіх військових операцій, місцем підготовки новобранців та центром командування. Втрата головної бази означала катастрофу для римського легіону, тому її захист був пріоритетним завданням. У той же час знищення ворожої бази було головною стратегічною метою в будь-якій військовій кампанії. Знищити базу – головна мета ворожої команди в програмі на тему Roman Conquest.

Ефективність римської армії значною мірою базувалася на взаємодії між різними рангами військовослужбовців. Як і в реальному житті, на війні панує співпраця між воїнами. Звичайні воїни не мають жодних особливих здібностей, проте вони, перебуваючи поруч із досвідченими преторіанцями, дивлячись на них, можуть отримати натхнення до битви, бути мотивованими, отримуючи підвищення

своїх характеристик поки перебувають у певній близькості з преторіанцем. Такий психологічний фактор відігравав важливу роль у підтриманні бойового духу. Центуріони, як зазначалось вище, можуть лагодити башти, щоб ті ще трохи допомогли у ході конфлікту. Ця система наставництва працювала природним чином та давала відчутні результати у бою. Сучасні технології дають змогу моделювати ці взаємодії через об'єктно-орієнтоване програмування, поєднуючи вивчення історії з демонстрацією принципів ООП.

1.2 Існуючі реалізації

На сьогодні існує чимало програмних продуктів, що присвячені тематиці Давнього Риму та його військовій історії. На цю тему є багато програмних реалізацій, проте кожна з них має свої особливості, переваги та недоліки. Однією з найвідоміших реалізацій є гра Total War Rome 2, розроблена студією Creative Assembly, де було реалізовано поєднання стратегії в реальному часі та покрокової стратегії із сетингом Давнього Риму. Цей проект поєднує елементи стратегії в реальному часі з покроковою стратегічною картою, де гравець може обирати одну з багатьох фракцій на основі історичних сценаріїв. Гра пропонує масштабні битви з тисячами воїнів на екрані, детальну систему управління провінціями та дипломатії, а також історичні кампанії, засновані на реальних подіях.

Іншим цікавим прикладом є гра Ryse Son of Rome, створена компанією Crytek, де гравець бере під своє керування простого легіонера, котрий з часом проходить підвищення до командувача. На відміну від стратегічного жанру попереднього проекту, ця гра належить до категорії action-adventure з видом від третьої особи. Гравець керує римським легіонером, який проходить шлях від звичайного солдата до командира високого рангу. Геймплей зосереджений на динамічних бойових сценах та системі швидкого реагування на дії противників. Гра відрізняється кінематографічною подачею та вражаючою графікою.

Аналізуючи ці реалізації, можна виділити як їхні сильні сторони, так і суттєві недоліки. Total War Rome 2 пропонує глибоку стратегічну складову та масштабність, проте висока складність механік може бути серйозним бар'єром для

початківців, які лише знайомляться з історією Риму. Система управління вимагає значного часу на освоєння, що може відлякати користувачів. Великі битви потребують потужного апаратного забезпечення для комфортної гри, що робить цей продукт недоступним для користувачів із звичайними комп'ютерами. Крім того, історична достовірність іноді приноситься в жертву ігровому балансу. Гра також є комерційним продуктом з високою ціною.

Ryse Son of Rome зосереджується на індивідуальному досвіді одного воїна, що дозволяє краще відчувати атмосферу. Однак лінійність сюжету та обмежена варіативність геймплею значно знижують реіграбельність проекту. Гра також критикується за спрощену бойову систему, де після освоєння базових механік процес стає досить повторюваним. Освітній аспект присутній переважно у формі довідкової інформації, яка не інтегрована органічно в ігровий процес.

Крім того, дана реалізація підлягає обмеженням PG через присутність моментів жорстокості, неприхованої крові та насильства, що є перешкодою для розповсюдження серед молодшого покоління. Обидва проекти мають високі системні вимоги та є платними продуктами, що обмежує доступ широкої аудиторії студентів.

З точки зору освітньої цінності, існуючі реалізації не завжди дотримуються оптимального балансу між розважальною та навчальною складовою. Також варто відзначити відсутність проектів, орієнтованих на демонстрацію принципів об'єктно-орієнтованого програмування через історичну тематику. Більшість навчальних програм з ООП використовують абстрактні приклади, упускаючи можливість поєднати програмування з історією. Така комбінація могла б підвищити мотивацію студентів до вивчення обох дисциплін одночасно. Враховуючи вищезазначені обмеження, моя реалізація є актуальною, оскільки пропонує безкоштовний та доступний інструмент для вивчення історії, який не вимагає потужного обладнання. Програма дозволить студентам бачити, як абстрактні класи відображають реальну ієрархію римського війська. Використання Java забезпечить платформонезалежність.

1.3 Розробка технічного завдання на роботу

1.4 Обґрунтування вибору мови програмування

Вибір мови програмування є критично важливим рішенням на етапі проектування будь-якого програмного продукту. Для реалізації проекту на тему Roman Conquest необхідно порівняти три основні мови програмування: C++, C# та Java, і довести, що для даної програми Java є оптимальним вибором. Кожна з цих мов має свої особливості, переваги та недоліки, які необхідно детально проаналізувати.

C++ є потужною мовою програмування, яка розвивалася з мови C шляхом додавання об'єктно-орієнтованих можливостей. Ця мова надає програмісту максимальний контроль над апаратними ресурсами, дозволяє працювати безпосередньо з пам'яттю через покажчики та пропонує високу продуктивність. Програми на C++ компілюються в машинний код для конкретної платформи, що забезпечує швидкість виконання, але вимагає окремої компіляції для кожної операційної системи.

C# розроблена компанією Microsoft як частина платформи .NET Framework та має багато спільного з Java за синтаксисом та концепціями. Ця мова тісно інтегрована з екосистемою Microsoft, що робить її відмінним вибором для розробки додатків під Windows. Протягом останніх років підтримка інших платформ покращилася завдяки проекту .NET Core, але історично C# мала обмеження щодо кросплатформенності.

Java є об'єктно-орієнтованою мовою програмування, розробленою компанією Sun Microsystems у середині дев'яностих років. Головною філософією Java є принцип «напиши один раз, запускай скрізь», що досягається завдяки використанню віртуальної машини Java. Програма, написана на Java, компілюється в байт-код, який потім інтерпретується віртуальною машиною незалежно від операційної системи. Це означає, що один і той же виконуваний файл може працювати на Windows, macOS, Linux без перекомпіляції.

Для детального порівняння цих мов доцільно розглянути їхні характеристики, наведені в таблиці 1.1.

Таблиця 1.1 – Порівняння мов програмування C++, C# та Java

Характеристика	C++	C#	Java
Платформонезалежність	Потребує окремої компіляції для кожної платформи	Обмежена, покращена в .NET Core	Повна підтримка через JVM
Управління пам'яттю	Ручне через покажчики	Автоматичне збирання сміття	Автоматичне збирання сміття
Швидкість розробки	Середня через складність синтаксису	Висока завдяки інструментам Microsoft	Висока завдяки бібліотекам
Складність вивчення	Висока через низькорівневі концепції	Середня, схожа на Java	Середня, зрозумілий синтаксис
Графічні бібліотеки	Qt, wxWidgets, SFML	WPF, Windows Forms	JavaFX, Swing
Багатопотоковість	Складніша реалізація	Вбудована підтримка	Вбудована підтримка

Аналізуючи специфіку проекту на тему Roman Conquest, можна виділити кілька ключових вимог. По-перше, програма має бути доступною для максимально широкої аудиторії, включаючи користувачів різних операційних систем. Студенти

можуть працювати на комп'ютерах з Windows, macOS або Linux, тому важливо забезпечити однакову функціональність на всіх платформах. По-друге, для навчального проекту важлива простота та зрозумілість коду. Java пропонує чистий синтаксис, де кожен клас зберігається в окремому файлі, що полегшує навігацію. C++ має складнішу структуру з заголовковими файлами, що може заплутати початківців.

По-третє, автоматичне управління пам'яттю є важливим фактором для навчального проекту. У Java та C# програмісту не потрібно вручну виділяти та звільняти пам'ять, оскільки це робить вбудований збирач сміття. У C++ необхідно самостійно контролювати час життя об'єктів, що вимагає досвіду. Для студентів, які починають вивчати об'єктно-орієнтоване програмування, автоматичне управління пам'яттю дозволяє зосередитися на логіці програми та принципах ООП. По-четверте, наявність потужної стандартної бібліотеки значно прискорює розробку. Java надає величезний набір готових класів для роботи з колекціями, файлами, графікою. Бібліотека JavaFX пропонує сучасний підхід до створення графічних інтерфейсів з підтримкою анімації.

По-п'яте, підтримка багатопотоковості є важливою для створення динамічної симуляції, де різні об'єкти повинні функціонувати одночасно. Java має вбудовану підтримку багатопотокового програмування з класами Thread, Runnable та пакетом `java.util.concurrent`. Це дозволяє реалізувати паралельне виконання логіки різних ігрових об'єктів без ускладнення коду. Що стосується C#, хоча ця мова має багато спільного з Java, вона традиційно орієнтована на екосистему Windows. Java залишається більш перевіреним рішенням для створення додатків, які працюють на різних платформах. Також варто враховувати, що багато навчальних закладів використовують Java як основну мову для викладання об'єктно-орієнтованого програмування.

Популярність мови також відіграє роль у виборі. Згідно з рейтингом TIOBE, який відстежує популярність мов програмування на основі кількості пошукових запитів, Java стабільно входить до трійки найпопулярніших мов програмування у світі. Це означає наявність великої спільноти розробників, величезної кількості

навчальних матеріалів, готових бібліотек та фреймворків, а також активну підтримку мови. Враховуючи всі вищезазначені фактори, Java є оптимальним вибором для реалізації проекту на тему Roman Conquest. Ця мова забезпечує необхідну платформонезалежність, має зрозумілий синтаксис для навчальних цілей, пропонує автоматичне управління пам'яттю, містить потужну бібліотеку JavaFX для створення графічного інтерфейсу, надає засоби для багатопотокового програмування та підтримується великою спільнотою. Усі ці переваги роблять Java ідеальним інструментом для створення освітнього програмного продукту.

1.5 Висновки

У першому розділі було проаналізовано предметну область проекту, що охоплює військову організацію Давнього Риму з її ієрархічною структурою. Розглянуто мікрооб'єкти (воїни, центуріони, преторіанці) та макрооб'єкти (джерела руди, башти, бази). Комплексний аналіз існуючих реалізацій виявив їхні недоліки: високі системні вимоги та значну вартість. Порівняння мов програмування C++, C# та Java дозволило обґрунтувати вибір Java як оптимального інструменту завдяки платформонезалежності, зрозумілому синтаксису та потужній бібліотеці JavaFX.

2 РОЗРОБКА ПІДСИСТЕМИ ГРАФІЧНОГО ВІДОБРАЖЕННЯ

2.2 Модель графічного відображення

Кожен мікрооб'єкт складається з власної PNG-картинки, прямокутника, що відображає рівень очок здоров'я мікрооб'єкта, та назва мікрооб'єкта.

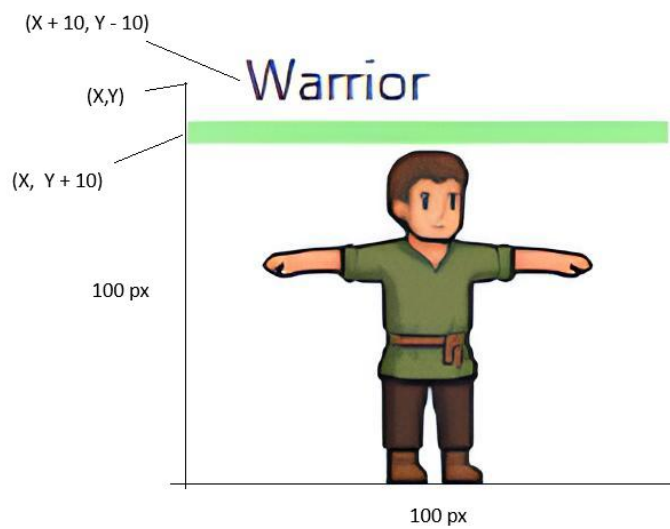


Рисунок 2.1 – Зображення мікрооб'єкта «Воїн»

(пізніше додам інші мікрооб'єкти)

Для вирахування мікрооб'єктів за основу було взято початкове положення зображення X та Y (див. *рисунок 2.1*), які є приватними полями у батьківському класі мікрооб'єкта. Коефіцієнти були підібрані так, щоб було чітко видно кожен елемент мікрооб'єкта.

(опис макрооб'єктів та самі макрооб'єкти додам пізніше, як будуть реалізовані в програмі)

2.3 Графічні процедури підсистеми графічного відображення

Для створення зображень мікро- та макрооб'єктів у програмі було використано графічні засоби бібліотеки JavaFX. Основними використаними графічними примітивами були:

`Group()` – конструктор, що створює групу, яка може містити у собі деяку кількість елементів класу `Node` для їх спільного розміщення на головний екран програми.

`Scene(Parent root, double width, double height)` – створює основну сцену розміром 1080×700 пікселів, яка містить `Group` з усіма графічними об'єктами.

`Image(String url)` – завантажує картинку із переданого URL-посилання або шляху у пам'яті комп'ютера.

`ImageView(Image image)` – конструктор елементу-картинки, який є наслідником класу `Node`, що дозволяє розміщати його на екрані.

`Label(String text)` – створює текстову мітку із заданим текстом.

`Line()` – створює лінійний елемент для відображення полоски здоров'я.

`Rectangle(double x, double y, double width, double height)` – створює прямокутник з координатами верхнього лівого кута (x, y), шириною та висотою.

`setCoordinates()` – переписаний методу класу `Unit`, який синхронізує всі графічні елементи (`Label`, `Line`, `ImageView`, `Rectangle`) з логічними координатами юніта.

`setPosition(double newX, double newY)` – встановлює нову позицію юніта на екрані.

`getBoundsInParent().contains(double x, double y)` – метод `ImageView`, який перевіряє, чи знаходиться точка з координатами (x, y) всередині меж зображення.

`AnimationTimer()` – конструктор елементу, який містить методи `start()`, `stop()` та переписаний метод `handle(long now)`.